

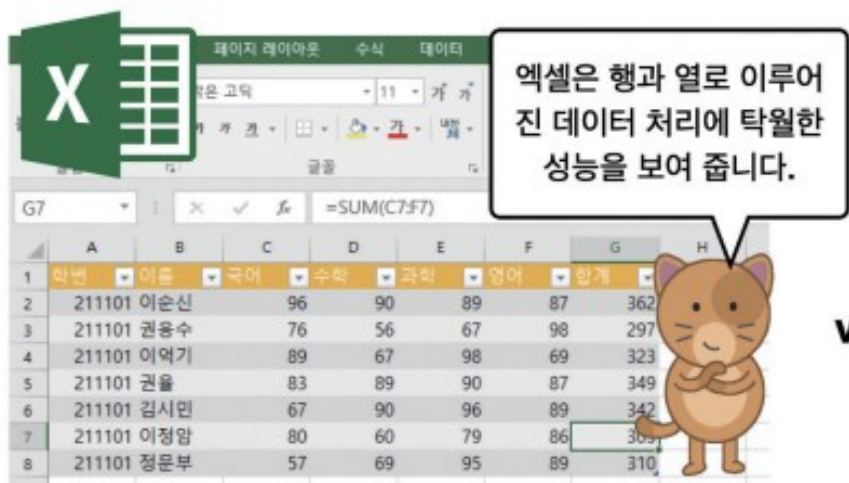
엑셀보다 판다스 Karl

Pandas



엑셀보다 빠르고 강력한 판다스

- 아래의 프로그램은 현재 업무용 소프트웨어로 가장 인기가 있는 마이크로소프트사의 **엑셀Excel**이라는 프로그램이다.
- 데이터 과학자는 전통적으로 이와 같은 테이블 형태의 데이터를 선호한다. 2차원 **행렬matrix** 형태의 데이터는 개발자가 편리하게 각 요소나 행, 열에 접근할 수 있기 때문이다.
- 파이썬의 **판다스pandas** 패키지를 사용하면 이러한 문제를 해결할 수 있다.



VS



엑셀보다 빠르고 강력한 판다스

- 판다스는 넘파이를 기반으로 하기 때문에 처리속도가 빠르고, 행과 열로 잘 구조화된 데이터프레임을 제공하며, 이 데이터프레임을 조작할 수 있는 다양한 함수를 지원한다.

판다스의 주요 특징

1. 빠르고 효율적이며 다양한 표현력을 갖춘 자료 구조

실세계 데이터 분석을 위해 만들어진 파이썬 패키지로 강력하면서도 다양한 표현력을 가지고 있다.

2. 다양한 형태의 데이터에 적합

이종 자료형의 열을 가진 테이블 데이터

시계열 데이터

레이블을 가진 다양한 행렬 데이터

다양한 관측 통계 데이터

3. 핵심 구조

시리즈Series: 1차원 구조를 가진 하나의 열

데이터프레임DataFrame: 복수의 열을 가진 2차원 데이터

4. 판다스가 잘하는 일

결측 데이터 처리와 필터링

테이블 형식의 데이터 추가와 삭제

열 데이터의 정렬과 다양한 데이터 조작

파이썬 리스트, 딕셔너리, 넘파이 배열을 데이터프레임으로 손쉽게 변환

데이터프레임의 각 필드들 사이의 상관 관계를 구하는 일

각 데이터프레임 열에서 결측값의 개수와 정상값의 수를 계산

맷플롯립과 잘 결합되어 손쉽게 데이터를 시각화

한 데이터프레임을 특정한 기준에 따라 여러 개의 데이터프레임으로 분할

두 데이터프레임을 결합하고 다시 인덱싱하는 능력

판다스는 여기에 열거한 기능 이외에도 데이터 처리와 분석을 위한 다양하면서도 강력한 기능을 제공하고 있다.

시리즈와 자료형과 결손값

- 앞 장에서 살펴본 넘파이의 2차원 행렬 데이터는 모두 같은 자료값을 가지는 단순한 수치 정보 중심으로만 구성되어 있다는 한계가 있다.
- 이 한계는 **판다스**`pandas` 패키지를 사용하여 극복할 수 있다.
- 넘파이의 다차원 배열과 같이 판다스는 **시리즈**와 **데이터프레임**이라는 두 가지의 기본적인고도 다재다능한 데이터 구조를 제공한다.

시리즈와 자료형과 결손값

- 판다스의 **시리즈series**는 같은 자료형의 데이터를 저장하는 인덱싱된 1차원 배열인데, 시리즈 데이터를 만드는 것은 Series 클래스를 이용한다.



```
import numpy as np
import pandas as pd

se = pd.Series([1, 2, np.nan, 4])  # np.nan은 결측값
se
```

```
0    1.0
1    2.0
2    NaN
3    4.0
dtype: float64
```

시리즈와 자료형과 결손값

- 데이터 과학을 위한 데이터를 다루게 될 경우, 잘 정제된 좋은 데이터보다는 정제되지 않은 데이터나 결측 데이터를 많이 다루게 된다.
- 이것을 **결손값** `missing value` 혹은 결측값이라 하는데 판다스는 이것을 탐지하고 수정하는 함수를 제공한다.
- 데이터에 **결손값**이 있는지를 **불리언** `boolean` 값으로 반환하는 함수는 `isna()`이다.

```
▶ se.isna() # np.nan이 포함된 세 번째 항목만 True를 반환
```

0	False
1	False
2	True
3	False

dtype: bool

시리즈와 자료형과 결손값

- 시리즈 자료구조 se의 첫 번째 원소와 두 번째 원소를 추출하기 위해서는 다음과 같은 인덱싱 기법을 사용한다.
- 이 방법은 리스트 인덱싱과 동일한 문법을 사용한다.

	<code>se[0], se[1]</code>
	<code>(1.0, 2.0)</code>

시리즈와 자료형과 결손값

- 앞서 살펴본 시리즈 se는 0, 1, 2, 3과 같은 인덱스를 가진 자료값이었는데 이 인덱스는 다음과 같은 방법을 사용하여 'a', 'b', 'c', 'd'와 같은 알파벳 문자로 변경할 수도 있다.



```
data = [1, 2, np.nan, 4]
indexed_se = pd.Series(data, index = ['a', 'b', 'c', 'd'])
print(indexed_se)
```

```
a    1.0
b    2.0
c    NaN
d    4.0
dtype: float64
```



```
indexed_se['a'], indexed_se['b'] # 인덱스가 'a', 'b',.. 임
```

```
(1.0, 2.0)
```

```
data = [1, 2, np.nan, 4]
pd.Series(data, index = ['a', 'b', 'c', 'd'])
```

	data
a	1.0
b	2.0
c	NaN
d	4.0

인덱스를 가진 시리즈의 정의 방법과 그 구조

판다스의 기본 구조인 시리즈와 데이터프레임

- 판다스에서는 각 월의 수익을 아래와 같이 딕셔너리 구조로 정의한 후, 이 딕셔너리를 이용하여 income_se 시리즈를 생성하는 코드를 만들 수 있다.



```
income = {'1월': 9500, '2월': 6200, '3월': 6050, '4월': 7000}
income_se = pd.Series(income) # 월을 인덱스로 하는 매출 시리즈
print('동윤이네 상점의 수익')
print(income_se)
```

동윤이네 상점의 수익

1월 9500

2월 6200

3월 6050

4월 7000

dtype: int64

판다스의 기본 구조인 시리즈와 데이터프레임

- 앞서 살펴본 동윤이네 상점의 수익 정보와 함께 **지출 정보**와 같은 새로운 정보가 더 필요한 경우를 생각해보자.
- 만일 이 상점에 매달 일정한 물건 구입 비용이 발생한 경우, 수익과 지출을 모두 표기하기 위해서는 **1차원의 시리즈 자료구조로는 표현이 어렵다.**
- 따라서 다음과 같이 expenses_se 시리즈 자료를 만들어서 새롭게 데이터를 만들어야 한다. 이때 사용되는 2차원 자료구조가 바로 **데이터프레임** `dataframe`이다.

판다스의 기본 구조인 시리즈와 데이터프레임

- 여기서 df를 이용하여 출력할 경우 주피터 노트북에서는 아래와 같이 표를 만들어 주며, print(df)를 사용할 경우 텍스트 형식의 출력을 하게 된다.

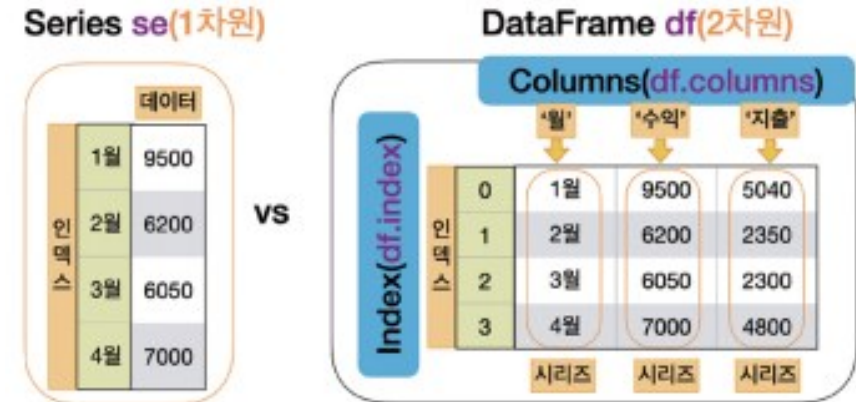


```
month_se = pd.Series(['1월', '2월', '3월', '4월'])
income_se = pd.Series([9500, 6200, 6050, 7000])
expenses_se = pd.Series([5040, 2350, 2300, 4800])
df = pd.DataFrame( {'월': month_se, '수익': income_se,
                    '지출' : expenses_se})
df      # print(df)를 이용해서 출력한 후 출력 형식을 비교해 보자
```

	월	수익	지출
0	1월	9500	5040
1	2월	6200	2350
2	3월	6050	2300
3	4월	7000	4800

판다스의 기본 구조인 시리즈와 데이터프레임

- 다음은 1차원 데이터인 시리즈와 2차원 데이터인 데이터프레임을 비교하는 것으로 왼쪽의 시리즈는 2차원처럼 보이지만 1월, 2월, 3월, 4월이 인덱스이므로 실제로는 **1차원 데이터**이다.



- 다음으로 income_se 시리즈의 최대값과 평균값을 출력해 보자.



```
# 판다스 Series를 이용하여 최대 수익 월을 출력하기
m_idx = np.argmax(income_se) # 넘파이의 argmax() 사용
print('최대 수익이 발생한 월:', month_se[m_idx])
```

최대 수익이 발생한 월: 1월



```
print('월 최대 수익:', income_se.max(),\
      ', 월 평균 수익:', income_se.mean())
```

월 최대 수익: 9500 , 월 평균 수익: 7187.5

판다스의 기본 구조인 시리즈와 데이터프레임



월 수입과 지출 계산하기

난이도 중

이 절의 df 데이터프레임을 이용하여 동윤이네 상점의 1월부터 4월까지 월 수입의 합과 월 지출의 합을 출력해보아라(힌트: [숫자 구분자를 출력하는 출력 포매터를 조사해 보자](#)).

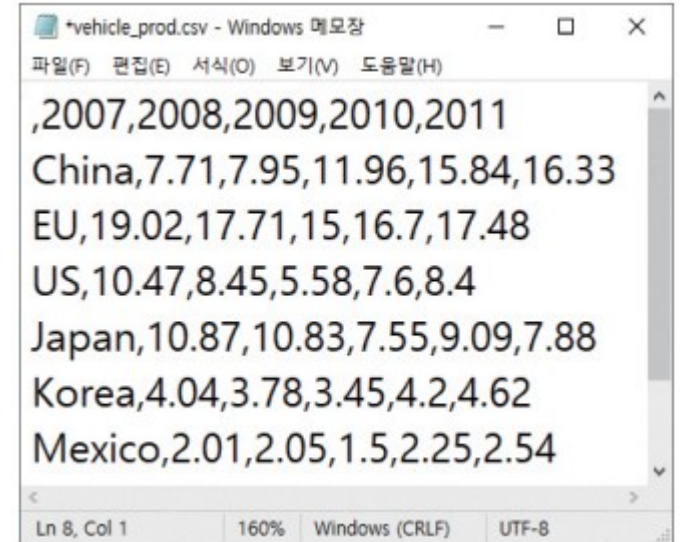
동윤이네 상점의 수입 합계: 28,750

동윤이네 상점의 지출 합계: 14,490

csv 데이터를 읽고 확인하기

- CSV는 쉼표로 구분한 변수 comma separated variables의 약자이다.
- 탭 tab 문자를 구분자로 사용하는 파일을 TSV라고 한다.

- 데이터의 중간에 쉼표(,)가 포함되어 있다. 이러한 경우에는 구분자로 사용되는 쉼표와 구분하기 위하여 반드시 데이터를 따옴표로 감싸야 한다.



- 앞서 살펴본 넘파이는 2차원 행렬 형태의 데이터를 지원하지만, 데이터의 속성을 표시하는 행이나 열의 레이블을 가지고 있지 않다. 따라서, 데이터의 속성을 한눈에 이해하기에는 불편 하다.

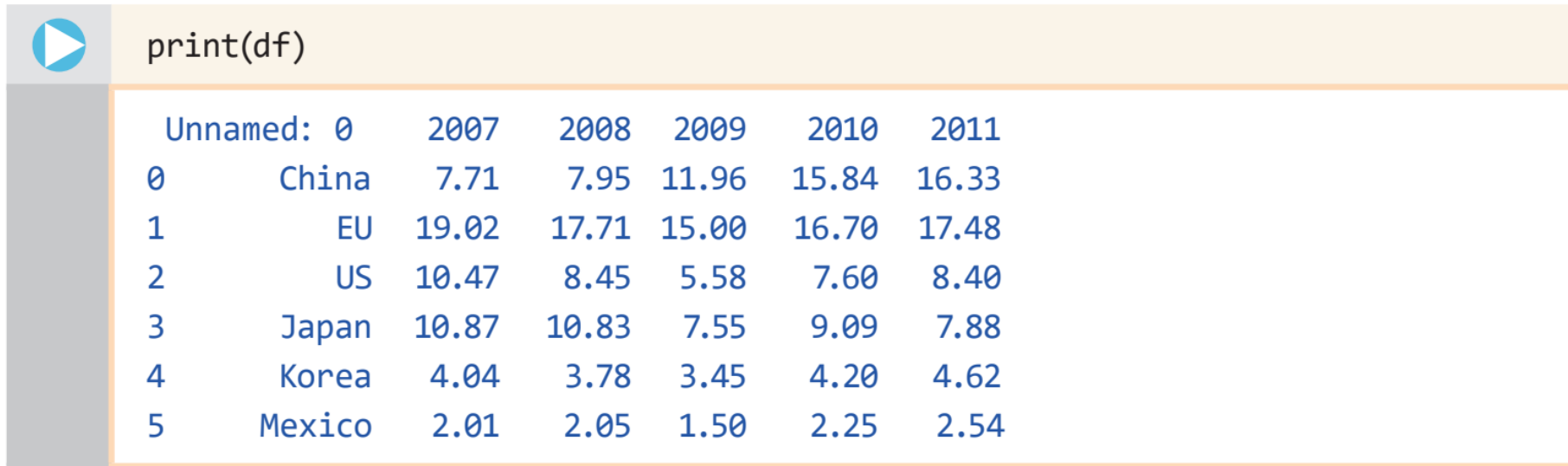
csv 데이터를 읽고 확인하기

- 다운 받은 vehicle_prodct.csv 파일을 실행할 폴더에 복사한다

```
file = './vehicle_prod.csv'  
df = pd.read_csv(file)
```

csv 데이터를 읽고 확인하기

- 이 파일에는 다음과 같이 국가에 대한 정보와 연도별 생산 대수가 저장되어 있는데, 첫 번째 열에는 인덱스, 두 번째는 국가명, 세 번째에서 다섯 번째 열은 2007년부터 2011년도 의 생산 대수가 백만 단위로 저장되어 있다.



A screenshot of a Jupyter Notebook cell. The cell has a blue play button icon and the text `print(df)`. Below the code, the output of the DataFrame is displayed as a table with 7 columns: 'Unnamed: 0', '2007', '2008', '2009', '2010', and '2011'. The rows represent different countries: China, EU, US, Japan, Korea, and Mexico, indexed from 0 to 5.

Unnamed: 0		2007	2008	2009	2010	2011
0	China	7.71	7.95	11.96	15.84	16.33
1	EU	19.02	17.71	15.00	16.70	17.48
2	US	10.47	8.45	5.58	7.60	8.40
3	Japan	10.87	10.83	7.55	9.09	7.88
4	Korea	4.04	3.78	3.45	4.20	4.62
5	Mexico	2.01	2.05	1.50	2.25	2.54

- 각 열들은 서로 다른 속성(property)을 나타낸다.

데이터프레임의 구조

- 데이터프레임에서는 다음과 같이 **인덱스**index와 **컬럼**columns 객체를 정의하여 사용한다.
- 판다스의 read_csv() 함수에 index_col이라는 키워드 매개변수에 인자 0을 넘겨주면 **첫 번째 열이 인덱스로** 사용된다.

csv 파일의 첫 행으로 만들어진 column

	Unnamed: 0	2007	2008	2009	2010	2011
0	China	7.71	7.95	11.96	15.84	16.33
1	EU	19.02	17.71	15	16.7	17.48
2	US	10.47	8.45	5.58	7.6	8.4
3	Japan	10.87	10.83	7.55	9.09	7.88
4	Korea	4.04	3.78	3.45	4.2	4.62
5	Mexico	2.01	2.05	1.5	2.25	2.54

자동으로 생성된 index



```
df = pd.read_csv(file, index_col = 0)
print(df)
```

	2007	2008	2009	2010	2011
China	7.71	7.95	11.96	15.84	16.33
EU	19.02	17.71	15.00	16.70	17.48
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88
Korea	4.04	3.78	3.45	4.20	4.62
Mexico	2.01	2.05	1.50	2.25	2.54

데이터프레임의 구조

- 이 데이터프레임의 컬럼값과 인덱스값을 출력하기 위해서는 다음과 같이 `columns`, `index` 속성을 사용하면 된다.



`df.columns` # 데이터프레임의 컬럼값들을 살펴보자

```
Index(['2007', '2008', '2009', '2010', '2011'], dtype='object')
```



`df.index` # 데이터프레임의 인덱스값들을 살펴보자

```
Index(['China', 'EU', 'US', 'Japan', 'Korea', 'Mexico'], dtype='object')
```

데이터프레임의 구조

- 데이터프레임에서 특정한 컬럼의 내용을 살펴보고자 할 경우 `df['2007']`과 같은 특정한 컬럼값을 인덱스로 사용하면 된다.

	<code>df['2007']</code> # 데이터프레임의 2007년도 컬럼값들을 살펴보자												
	<table><tr><td>China</td><td>7.71</td></tr><tr><td>EU</td><td>19.02</td></tr><tr><td>US</td><td>10.47</td></tr><tr><td>Japan</td><td>10.87</td></tr><tr><td>Korea</td><td>4.04</td></tr><tr><td>Mexico</td><td>2.01</td></tr></table> Name: 2007, dtype: float64	China	7.71	EU	19.02	US	10.47	Japan	10.87	Korea	4.04	Mexico	2.01
China	7.71												
EU	19.02												
US	10.47												
Japan	10.87												
Korea	4.04												
Mexico	2.01												
	<code>df.columns.tolist()</code> # 데이터프레임의 컬럼들을 리스트형으로 반환함												
	<code>['2007', '2008', '2009', '2010', '2011']</code>												
	<code>df['2007'].tolist()</code> # 2007년도 컬럼들의 값들을 리스트형으로 반환함												
	<code>[7.71, 19.02, 10.47, 10.87, 4.04, 2.01]</code>												

새로운 열을 생성하자

- 이 표에서 모든 레코드의 값을 합하는 $df['2007'] + df['2008'] + df['2009'] + df['2010'] + df['2011']$ 와 같은 표현은 $df.sum(axis = 1)$ 과 같은 표현과 동일하다.
- 여기서 $axis = 1$ 은 행 방향을 가리키며, $axis = 0$ 은 열 방향을 가리킨다.



```
df['total'] = df.sum(axis = 1)
print(df)
```

	2007	2008	2009	2010	2011	total
China	7.71	7.95	11.96	15.84	16.33	59.79
EU	19.02	17.71	15.00	16.70	17.48	85.91
US	10.47	8.45	5.58	7.60	8.40	40.50
Japan	10.87	10.83	7.55	9.09	7.88	46.22
Korea	4.04	3.78	3.45	4.20	4.62	20.09
Mexico	2.01	2.05	1.50	2.25	2.54	10.35

새로운 열을 생성하자

- 이 예제처럼 다음과 같은 방법으로 특정한 열을 인덱싱 한 후, `sum(axis=1)`, `mean(axis=1)`과 같이 메소드의 이름과 연산이 수행되는 축을 명시하여 5년간 자동차 생산 대수의 합과 평균을 구할 수도 있다.



```
df['total'] = df[['2007', '2008', '2009', '2010', '2011']].sum(axis=1)
df['mean'] = df[['2007', '2008', '2009', '2010', '2011']].mean(axis=1)
print(df)
```

	2007	2008	2009	2010	2011	total	mean
China	7.71	7.95	11.96	15.84	16.33	59.79	11.958
EU	19.02	17.71	15.00	16.70	17.48	85.91	17.182
US	10.47	8.45	5.58	7.60	8.40	40.50	8.100
Japan	10.87	10.83	7.55	9.09	7.88	46.22	9.244
Korea	4.04	3.78	3.45	4.20	4.62	20.09	4.018
Mexico	2.01	2.05	1.50	2.25	2.54	10.35	2.070

새로운 열을 생성하자



```
df.drop('2007', inplace=True, axis=1)
print(df)          # 이전에 구한 total, mean 값이 나타남
```

	2008	2009	2010	2011	total	mean
China	7.95	11.96	15.84	16.33	59.79	11.958
EU	17.71	15.00	16.70	17.48	85.91	17.182
US	8.45	5.58	7.60	8.40	40.50	8.100
Japan	10.83	7.55	9.09	7.88	46.22	9.244
Korea	3.78	3.45	4.20	4.62	20.09	4.018
Mexico	2.05	1.50	2.25	2.54	10.35	2.070



```
# 2008년부터 2011년도까지의 총생산 대수와 평균을 다시 계산함
df['total'] = df[['2008', '2009', '2010', '2011']].sum(axis=1)
df['mean'] = df[['2008', '2009', '2010', '2011']].mean(axis=1)
print(df)          # 새로 계산한 total, mean 값이 나타남
```

	2008	2009	2010	2011	total	mean
China	7.95	11.96	15.84	16.33	52.08	13.0200
EU	17.71	15.00	16.70	17.48	66.89	16.7225
US	8.45	5.58	7.60	8.40	30.03	7.5075
Japan	10.83	7.55	9.09	7.88	35.35	8.8375
Korea	3.78	3.45	4.20	4.62	16.05	4.0125
Mexico	2.05	1.50	2.25	2.54	8.34	2.0850

새로운 열을 생성하자



연도별 생산 대수의 합을 출력하자

난이도상

이 데이터프레임 `df`를 이용하여 연도별 생산 대수의 합을 다음과 같이 `total` 이라는 이름의 행으로 가장 아래 행에 출력해 보자(힌트: 데이터프레임에서 국가명을 제외한 숫자의 합을 구하는 메소드로 `select_dtypes(np.number).sum().rename()` 이 있다).

	2007	2008	2009	2010	2011
China	7.71	7.95	11.96	15.84	16.33
EU	19.02	17.71	15.00	16.70	17.48
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88
Korea	4.04	3.78	3.45	4.20	4.62
Mexico	2.01	2.05	1.50	2.25	2.54
Total	54.12	50.77	45.04	55.68	57.25

inplace로 데이터프레임 갱신하기

- 이 옵션이 True이면 명령어를 실행한 후 메소드가 적용된 기존의 데이터프레임을 갱신하게 되며, False이면 기존 데이터프레임은 그대로 두고 갱신된 데이터프레임을 반환하게 된다.

```
d_df = pd.DataFrame(data = [[10, 20, 30, 40], [50, 60, 70, 80]],  
                     columns = ['A', 'B', 'C', 'D'])  
new_df = d_df.drop('B', axis=1, inplace=False)  
print(new_df)      # 'B'열이 삭제된 새로운 데이터프레임
```

	A	C	D
0	10	30	40
1	50	70	80

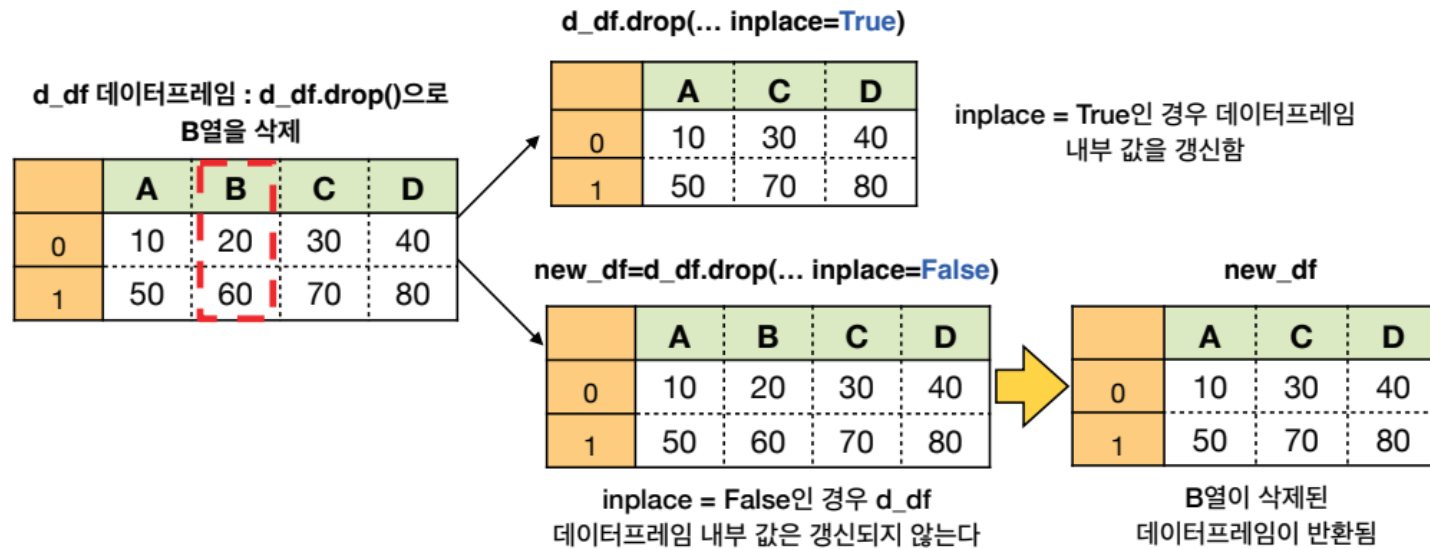
```
print(d_df)      # d_df 데이터프레임은 변화가 없음
```

	A	B	C	D
0	10	20	30	40
1	50	60	70	80

```
d_df.drop('B', axis=1, inplace=True) # inplace 키워드 인자가 True  
print(d_df)      # d_df 데이터프레임 내부의 값이 변경됨
```

	A	C	D
0	10	30	40
1	50	70	80

inplace로 데이터프레임 갱신하기



- inplace가 True일 경우 d_df 데이터프레임 자체의 값이 변경되므로 'B' 열이 완전히 없어지는 것을 볼 수 있다.

inplace로 데이터프레임 갱신하기

- inplace=True로 하여 df 데이터프레임을 갱신하는 방식을 사용해 보자.
- 이를 위해서 axis = 0으로 축을 명시해야 한다는 것에 유의하도록 하자.

```
▶ file = './vehicle_prod.csv'  
df = pd.read_csv(file, index_col=0)  
df['total'] = df.sum(axis=1)  
print(df)
```

	2007	2008	2009	2010	2011
China	7.71	7.95	11.96	15.84	16.33
EU	19.02	17.71	15.00	16.70	17.48
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88
Korea	4.04	3.78	3.45	4.20	4.62
Mexico	2.01	2.05	1.50	2.25	2.54

```
▶ df.drop('Mexico', axis=0, inplace=True) # Mexico행을 삭제하고 df를 갱신  
print(df)
```

	2007	2008	2009	2010	2011
China	7.71	7.95	11.96	15.84	16.33
EU	19.02	17.71	15.00	16.70	17.48
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88
Korea	4.04	3.78	3.45	4.20	4.62

데이터프레임 시각화

- 두 데이터프레임에서 2007년도 레이블을 가진 열만을 추출하기 위해서는 원하는 컬럼의 레이블 ['2007']을 사용하면 된다.



```
df = pd.read_csv(file, index_col = 0)
df_no_index = pd.read_csv(file)
print(df['2007'])
print(df_no_index['2007'])
```

```
China      7.71
EU         19.02
US         10.47
Japan      10.87
Korea       4.04
Mexico      2.01
Name: 2007, dtype: float64
0         7.71
1        19.02
2        10.47
3        10.87
4         4.04
5         2.01
Name: 2007, dtype: float64
```

데이터프레임 시각화

- 주의해서 볼 점은 `index_col = 0`과 같이 첫 열을 인덱스로 지정한 경우 국가 코드가 인덱스로 사용되기 때문에 국가 코드가 표시되고 있다는 점이며, 별도의 `index_col`을 지정하지 않을 경우 0부터 5까지의 인덱스를 자동으로 생성하여 인덱싱을 한다는 점이다.



```
print(df[['2007', '2008', '2009']])
```

	2007	2008	2009
China	7.71	7.95	11.96
EU	19.02	17.71	15.00
US	10.47	8.45	5.58
Japan	10.87	10.83	7.55
Korea	4.04	3.78	3.45
Mexico	2.01	2.05	1.50

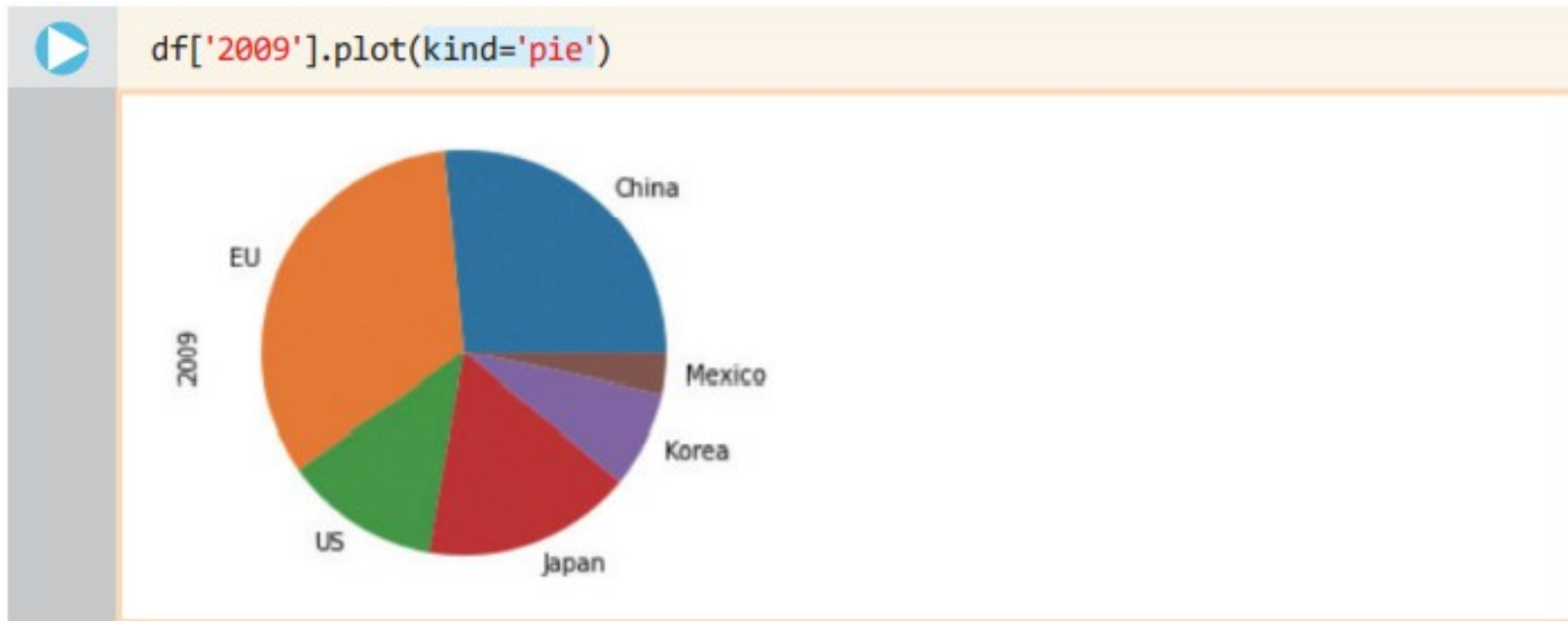
데이터프레임 시각화

- 우리는 다음과 같은 방법으로 선택된 열을 쉽게 그래프로 그릴 수 있다. 이를 위하여 데이터프레임의 `df['2009']`을 통해서 2009년도 열을 추출하고 `plot()` 메소드만 추가하면 된다.



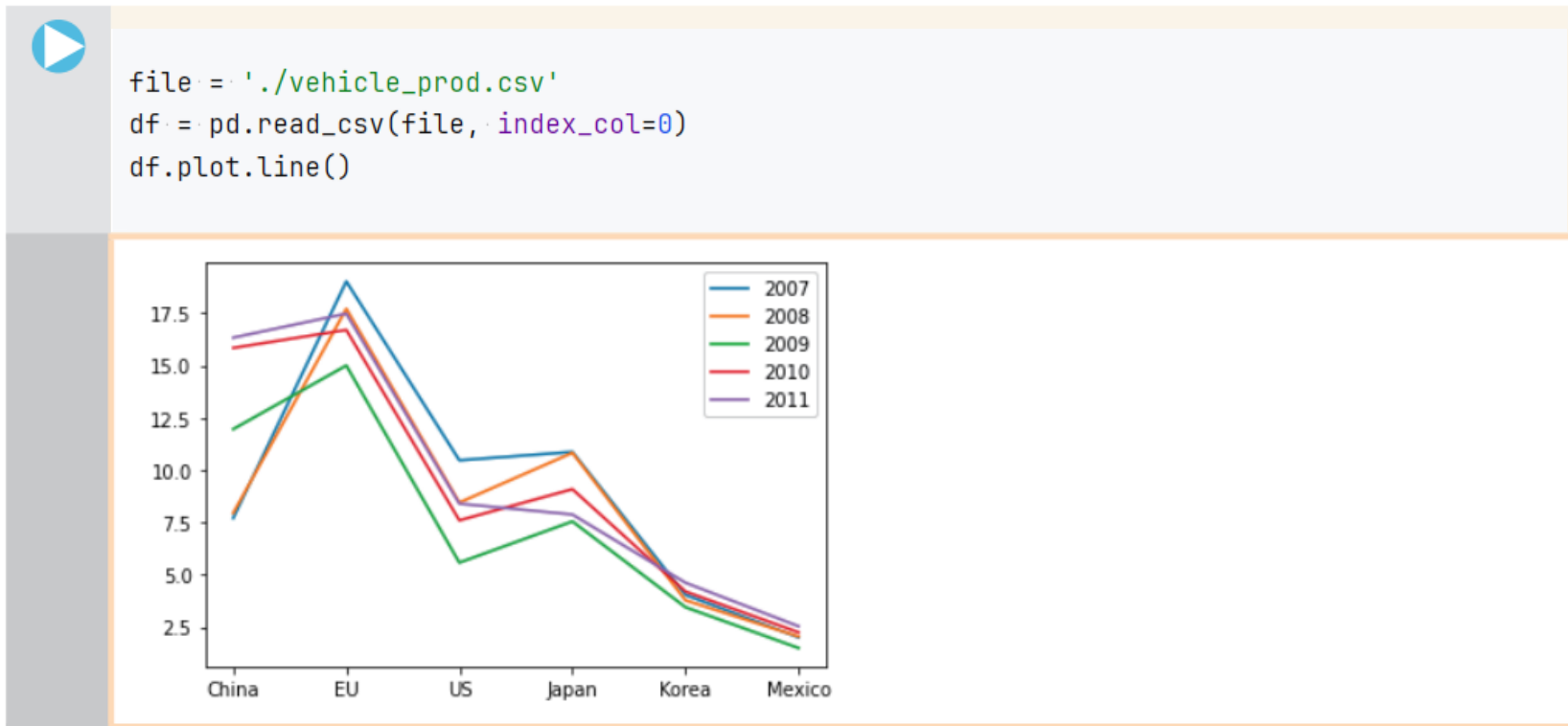
데이터프레임 시각화

- 코드에서 plot 메소드의 키워드 매개변수 kind는 차트의 종류를 의미한다.
- 이제 다음과 같이 kind='pie'로 지정하고, color 키워드 인자 부분을 삭제하면 아래 그림처럼 파이 차트가 그려진다.



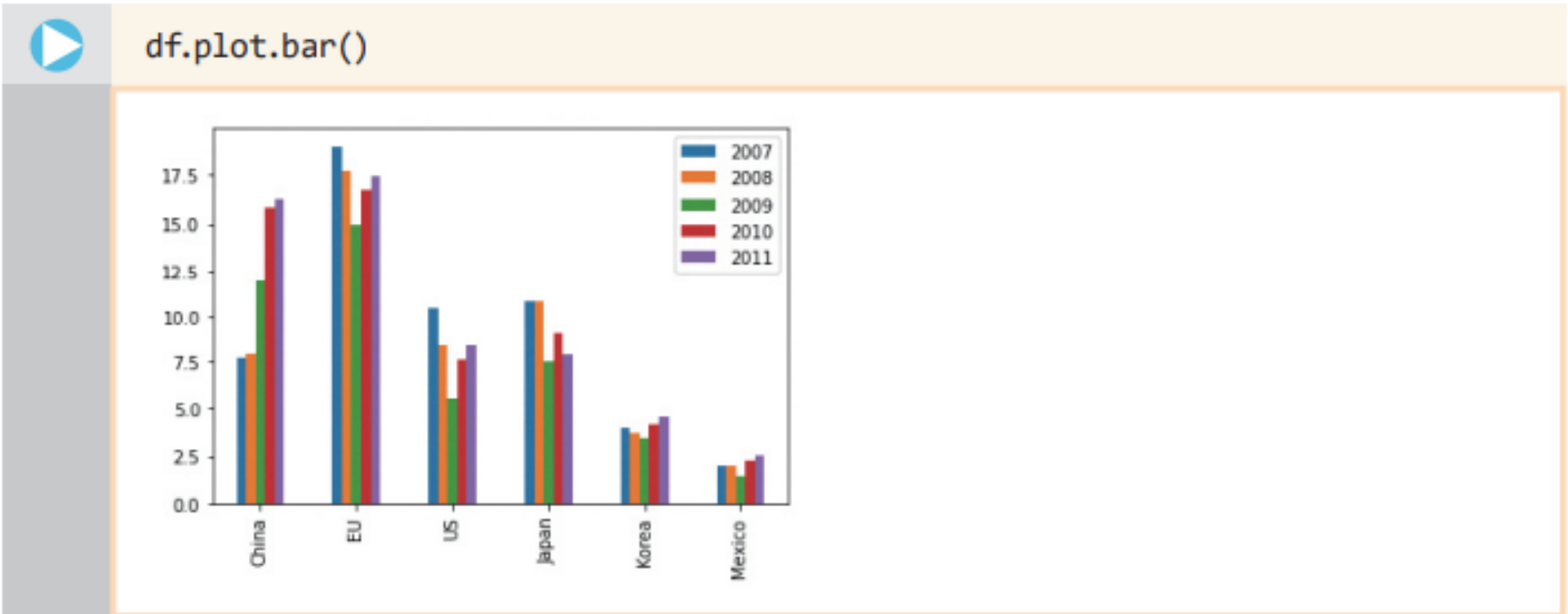
편리하고 강력한 시각화

- 데이터프레임은 맷플롯립과 결합되어 있어서 자료값을 손쉽게 시각화하여 보여줄 수 있다.



편리하고 강력한 시각화

- 선 그래프가 아닌 막대 그래프로 보고자 할 경우 `df.plot.bar()`를 호출하면 된다.



편리하고 강력한 시각화

- 이 선 그래프를 통해서 중국의 자동차 생산량이 매년 가파르게 증가하고 있으며, 일본의 경우 완만하게 감소하는 것을 이해할 수 있을 것이다.

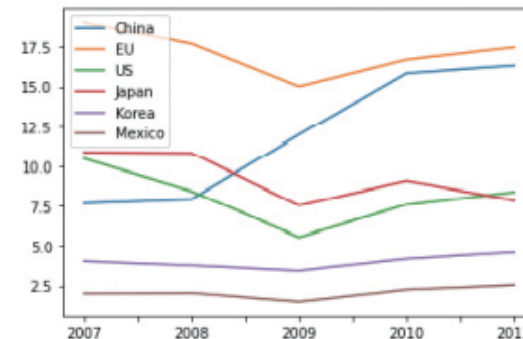


```
df = df.transpose() # df.T도 가능함  
print(df)
```

	China	EU	US	Japan	Korea	Mexico
2007	7.71	19.02	10.47	10.87	4.04	2.01
2008	7.95	17.71	8.45	10.83	3.78	2.05
2009	11.96	15.00	5.58	7.55	3.45	1.50
2010	15.84	16.70	7.60	9.09	4.20	2.25
2011	16.33	17.48	8.40	7.88	4.62	2.54



```
df.plot.line()
```



편리한 데이터 다루기 - 슬라이싱과 인덱싱

- 우선 처음 5행만 얻으려면 `head()`를 사용할 수 있다. 그리고, 마지막 5행만을 얻으려면 `tail()`을 사용한다. 만일 `head()`와 `tail()`에 정수를 인자로 넘겨주면 그 정수만큼의 행을 보여준다.



```
df = pd.read_csv(file, index_col=0)  # 원격지에 접속하여 csv를 읽어옴  
df.head(3)                          # 첫 3행의 정보를 가져온다
```

	2007	2008	2009	2010	2011
China	7.71	7.95	11.96	15.84	16.33
EU	19.02	17.71	15.00	16.70	17.48
US	10.47	8.45	5.58	7.60	8.40

편리한 데이터 다루기 - 슬라이싱과 인덱싱

- 여기서는 행의 개수가 3개뿐이어서 특별한 장점이 없어 보이지만, 행의 개수가 아주 많은 경우에는 `head()`와 `tail()`이 도움이 된다. 더욱 일반적인 방법은 다음과 같이 슬라이싱 표기법을 사용하는 것이다.



df[2:6]

지정된 범위의 레코드를 가져온다

	2007	2008	2009	2010	2011
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88
Korea	4.04	3.78	3.45	4.20	4.62
Mexico	2.01	2.05	1.50	2.25	2.54

편리한 데이터 다루기 - 슬라이싱과 인덱싱

- 만약 'Korea' 행을 선택하려면 `df.loc['Korea']`와 같이 큰 괄호를 사용하여 참조하려는 인덱스 이름을 적어주면 된다.



df.loc['Korea']	
2007	4.04
2008	3.78
2009	3.45
2010	4.20
2011	4.62
Name: Korea, dtype: float64	

편리한 데이터 다루기 - 슬라이싱과 인덱싱

- 그리고, 'US','Korea' 인덱스를 사용할 경우에는 다음과 같이 `[..]` 내부에 인덱스의 이름을 적어주어야 하며, `df.loc['US','Korea']`와 접근할 경우 키 오류가 발생하니 주의하도록 하자.



`df.loc[['US','Korea']]` # 주의 : `df.loc['US','Korea']`는 오류

	2007	2008	2009	2010	2011
US	10.47	8.45	5.58	7.6	8.40
Korea	4.04	3.78	3.45	4.2	4.62

편리한 데이터 다루기 - 슬라이싱과 인덱싱

- 다음으로는 열 선택과 행 선택을 결합하는 방법을 알아보자.

```
df['2011'][[0, 4]]
```

China	16.33
Korea	4.62

Name: 2011, dtype: float64

- 보다 상세하게 데이터프레임에서 특정한 요소 하나만을 선택하려면 간단하게 `loc()` 함수에 행 과 열의 레이블을 써주면 된다.

```
df.loc['Korea', '2011']
```

4.62

loc, iloc 인덱서

1. head(), tail() 메소드 : 첫, 마지막 항목 중에서 지정된 개수를 가져올 수 있다.
2. [m:n]을 사용 : 지정된 구간 m, n의 항목을 가져올 수 있다.
3. loc[] : 인덱스에서 특정 레이블이 있는 행을 가져온다.
4. iloc[] : 인덱스에서 정수형 인덱스를 사용하여 특정 위치에 있는 행을 가져온다.

loc, iloc 인덱서

- 데이터프레임의 행 데이터를 가져오기 위해서는 입력된 문서에 대하여 색인을 자동으로 작성해 주는 프로그램이 필요한데 이를 **인덱서** *indexer*라고 한다



```
df = pd.read_csv(file, index_col=0) # 원격지에 접속하여 csv를 읽어옴  
df.head(3)['2009'] # 첫 3행의 정보를 가져온다, 2009년도 값만을 가져온다
```

```
China    11.96  
EU       15.00  
US        5.58  
Name: 2009, dtype: float64
```


loc, iloc 인덱서

- 여기서는 전체 데이터 중에서 가장 앞에 있는 3개를 가져와서, 2009년도 열의 값을 추출하였다.

Unnamed: 0	2007	2008	2009	2010	2011
China	7.71	7.95	11.96	15.84	16.33
EU	19.02	17.71	15	16.7	17.48
US	10.47	8.45	5.58	7.6	8.4
Japan	10.87	10.87	10.87	10.87	10.87
Korea	4.04	3.78	3.45	4.2	4.62
Mexico	2.01	2.05	1.5	2.25	2.54

df.head(3)

df.head(3)['2009']

- 다음과 같이 df.iloc[4]을 사용할 경우 Korea를 인덱스로 하는 항목을 다음과 같은 열 형식으로 출력해 볼 수 있다.

```
df.iloc[4] # df.loc['Korea'] 와 동일함, df.iloc['Korea']는 오류
```

```
2007    4.04
2008    3.78
2009    3.45
2010    4.20
2011    4.62
Name: Korea, dtype: float64
```

loc, iloc 인덱서

- `iloc[]`는 정수형 인덱스만을 사용할 수 있기 때문에 `iloc['Korea']`를 사용할 경우 오류가 발생한다.



```
df.iloc[[2, 4]] # 주의: df.iloc[2, 4]는 US행의 2011년 데이터임
```

	2007	2008	2009	2010	2011
US	10.47	8.45	5.58	7.6	8.40
Korea	4.04	3.78	3.45	4.2	4.62



```
df.iloc[2, 4] # US행의 2011년 데이터를 출력
```

8.4

loc, iloc 인덱서



```
df.iloc[2:4]
```

```
# US행, Japan행의 데이터를 출력
```

	2007	2008	2009	2010	2011
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88



```
df.iloc[[2, 3]]
```

```
# US행, Japan행의 데이터를 출력
```

	2007	2008	2009	2010	2011
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88



```
df[2:4]
```

```
# US행, Japan행의 데이터를 출력
```

	2007	2008	2009	2010	2011
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88

판다스를 이용한 기상 데이터 분석

- 데이터프레임이 저장한 데이터를 간단히 분석하려면 `describe()` 함수를 사용할 수 있다.
- 우선 다음과 같은 코드로 데이터를 읽어 보자. 이 파일에는 한글이 포함되어 있으므로 `encoding='CP949'`를 통해 한글 인코딩 방식을 알려주도록 하자.



```
weather_file = './weather.csv'
```

```
weather = pd.read_csv(weather_file, index_col=0, encoding='CP949')
```

```
print(weather.head(3))
```

```
print('weather 데이터의 shape :', weather.shape)
```

	평균기온	최대풍속	평균풍속
일시			
2010-08-01	28.7	8.3	3.4
2010-08-02	25.2	8.7	3.8
2010-08-03	22.1	6.3	2.9

weather 데이터의 shape : (3653, 3)

판다스를 이용한 기상 데이터 분석

- 이제 데이터가 담고 있는 내용을 간략히 분석하기 위해 describe() 함수를 호출해 보자. count, 즉 각 특징별 총샘플수가 같지 않음을 확인한다.
- NaN이 하나라도 포함된 행을 모두 출력해보자.



```
print(weather.describe())
```

	평균기온	최대풍속	평균풍속
count	3653.000000	3649.000000	3647.000000
mean	12.942102	7.911099	3.936441
std	8.538507	3.029862	1.888473
min	-9.000000	2.000000	0.200000
25%	5.400000	5.700000	2.500000
50%	13.800000	7.600000	3.600000
75%	20.100000	9.700000	5.000000
max	31.300000	26.000000	14.900000

```
weather[weather.isna().any(axis=1)]
```

	평균기온	최대풍속	평균풍속
일시			
2012-02-11	-0.7	NaN	NaN
2012-02-12	0.4	NaN	NaN
2012-02-13	4.0	NaN	NaN
2015-03-22	10.1	11.6	NaN
2015-04-01	7.3	12.1	NaN
2019-04-18	15.7	11.7	NaN
2019-04-19	7.8	NaN	2.3

판다스를 이용한 기상 데이터 분석

- describe() 메소드를 통해서 각 속성값들의 **개수**^{count}, **평균값**^{mean}, **표준편차**^{std}, **최소값**^{min}, **최대값**^{max} 등을 쉽게 알 수 있다.
- 만약 문자열을 담고 있는 열이 있다면 해당 열은 분석에서 제외된다.
- 이 기능은 mean(), std() 함수를 통해 특정한 분석값만 볼 수도 있다.



```
print('평균 분석 -----')
print(weather.mean())
print('표준편차 분석 -----')
print(weather.std())
```

```
평균 분석 -----
평균기온      12.942102
최대풍속      7.911099
평균풍속      3.936441
dtype: float64
표준편차 분석 -----
평균기온      8.538507
최대풍속      3.029862
평균풍속      1.888473
dtype: float64
```

판다스를 이용한 기상 데이터 분석

- 판다스의 표준편차는 디폴트로 **베셀 보정** Bessel's correction을 적용하는데, 이것은 표준편차를 구할 때, 표본 크기 n 대신에 $n-1$ 을 적용하는 것으로 모분산 추정에서 편향을 보정하는 역할을 한다.



최대풍속, 평균풍속의 최대값 구하기

난이도 하

다음과 같이 전체 데이터에 대하여 최대풍속과 평균풍속의 최대값 26.0 m/s, 14.9 m/s을 각각 출력해 보도록 하자.

최대풍속의 최대값	26.0
평균풍속의 최대값	14.9

데이터 정제와 결손값의 처리

- 데이터를 처리하기 전에 반드시 거쳐야 하는 절차가 데이터 정제이다.
- 앞서 살펴본 shape 속성을 통해 전체 테이블의 크기를 살펴 볼 수 있지만, 개별적인 열의 개수를 알고 싶다면 count() 메소드를 사용할 수 있다.

```
▶ weather.count() # 개별적인 열의 개수를 알 수 있다
```

평균기온	3653
최대풍속	3649
평균풍속	3647
dtype:	int64

데이터 정제와 결손값의 처리

- 어떤 이유로 인해 날씨 데이터를 수집하는 중에 값이 입력되지 못하면 그 항목이 비어 있을 수 있는데 이를 **결손값** `missing data`이라고 한다.
- 결손값**을 탐지하는 함수를 이미 살펴보았다. 데이터에 결손값이 있는지를 **불리언** `boolean` 값으로 반환하는 메소드는 `isna()`이다. 평균풍속 열에 NaN이 포함된 모든 행을 출력해본다.



```
missing_data = weather[ weather['평균풍속'].isna() ]  
print(missing_data)
```

	평균기온	최대풍속	평균풍속
일시			
2012-02-11	-0.7	NaN	NaN
2012-02-12	0.4	NaN	NaN
2012-02-13	4.0	NaN	NaN
2015-03-22	10.1	11.6	NaN
2015-04-01	7.3	12.1	NaN
2019-04-18	15.7	11.7	NaN

데이터 정제와 결손값의 처리

- 각 열의 결손값^{missing data}의 개수를 세기 위해 `isna().sum()` 을 사용
- 전체 열의 결손값의 개수를 셀 때는 `isna().sum().sum()` 을 사용

weather[weather.isna().any(axis=1)]			
일시	평균기온	최대풍속	평균풍속
2012-02-11	-0.7	NaN	NaN
2012-02-12	0.4	NaN	NaN
2012-02-13	4.0	NaN	NaN
2015-03-22	10.1	11.6	NaN
2015-04-01	7.3	12.1	NaN
2019-04-18	15.7	11.7	NaN
2019-04-19	7.8	NaN	2.3

```
missing_count_mean = weather['평균풍속'].isna().sum()
missing_count_max = weather['최대풍속'].isna().sum()
missing_count_all = weather.isna().sum().sum()
print("평균풍속 결측치 갯수:", missing_count_mean)
print("최대풍속 결측치 갯수:", missing_count_max)
print("전체 결측치 갯수:", missing_count_all)
```

평균풍속 결측치 갯수: 6
최대풍속 결측치 갯수: 4
전체 결측치 갯수: 10

데이터 정제와 결손값의 처리

- 결손값을 다루는 가장 간단한 방법은 결손값을 가진 행이나 열 전체를 삭제하는 것이다.

축이 0이면 결손 데이터를 포함한 행을 삭제하고
축이 1이면 결손 데이터를 포함한 열을 삭제한다.

inplace가 True이면 원본 데이터에서 결손 데이터를 삭제하고
False인 경우는 원본은 그대로 두고 고쳐진 데이터프레임 반환

pandas.DataFrame.dropna(axis=0, how='any', inplace=False)

how의 값이 'any'이면 결손 데이터가 하나라도 포함되면 제거 대상이 되고,
'all'이면 axis 인자에 따라서 행 혹은 열 전체가 결손 데이터이어야 제거한다.



데이터 정제와 결손값의 처리

- 데이터셋을 받아온 후 아래 코드로 결손값이 포함된 행 제거

```
weather.dropna(axis=0, how='any', inplace = True)
missing_data = weather[ weather['평균풍속'].isna() ]

print(missing_data)
print(weather[weather.isnull().any(axis=1)])
weather.describe()
```

Empty DataFrame
Columns: [평균기온, 최대풍속, 평균풍속]
Index: []
Empty DataFrame
Columns: [평균기온, 최대풍속, 평균풍속]
Index: []

	평균기온	최대풍속	평균풍속
count	3646.000000	3646.000000	3646.000000
mean	12.954717	7.907899	3.936890
std	8.538707	3.029047	1.888538
min	-9.000000	2.000000	0.200000
25%	5.400000	5.700000	2.500000
50%	13.800000	7.600000	3.600000
75%	20.100000	9.700000	5.000000
max	31.300000	26.000000	14.900000

데이터 정제와 결손값의 처리

- 결손값을 다루는 또 하나의 방법은 결손값을 다른 값으로 교체하는 것이다. `fillna()` 메소드를 사용하면 특정한 값을 다른 값으로 교체한다.



```
# 결손값을 0으로 채움, inplace를 True로 설정해 원본 데이터를 수정
weather.fillna(0, inplace = True)
print(weather.loc['2012-02-11'])
```

```
평균기온    -0.7
최대풍속     0.0
평균풍속     0.0
Name: 2012-02-11, dtype: float64
```

weather.describe()

	평균기온	최대풍속	평균풍속
count	3653.000000	3653.000000	3653.000000
mean	12.942102	7.902436	3.929975
std	8.538507	3.039488	1.893644
min	-9.000000	0.000000	0.000000
25%	5.400000	5.600000	2.500000
50%	13.800000	7.600000	3.600000
75%	20.100000	9.700000	5.000000
max	31.300000	26.000000	14.900000

데이터 정제와 결손값의 처리

- 결손값을 단순히 0으로 채우게 되면 데이터의 개수가 적을 경우 전체 데이터의 대표값인 **평균값**을 왜곡할 수 있다.
- 열의 평균치를 구한다. 그리고 이 값으로 결손값을 채우면 다음과 같이 측정이 누락된 2012년 2월 11일의 풍속을 **전체 데이터의 평균**으로 채울 수 있다.



```
# 이전 코드는 fillna(0,..)을 통해 0으로 채움, 다음은 평균으로 채움
# 이전 코드 실행 후 아래 코드를 실행하면 이전 코드의 영향을 받음
# 커널 재시작을 하고 실행할 것
weather.fillna( weather['평균풍속'].mean(), inplace = True)
print(weather.loc['2012-02-11'])
```

```
평균기온    -0.7
최대풍속     3.936441
평균풍속     3.936441
Name: 2012-02-11, dtype: float64
```

	평균기온	최대풍속	평균풍속
count	3653.000000	3653.000000	3653.000000
mean	12.942102	7.906747	3.936441
std	8.538507	3.031055	1.886922
min	-9.000000	2.000000	0.200000
25%	5.400000	5.600000	2.500000
50%	13.800000	7.600000	3.600000
75%	20.100000	9.700000	5.000000
max	31.300000	26.000000	14.900000

데이터 정제와 결손값의 처리

- 시계열 데이터의 경우 결손값을 NaN이 아닌 직전의 값이나 직후의 값으로 채울 수 있다.
- 직전의 값으로 NaN을 채울 때는 `fillna(method='ffill')`
- 직후의 값을 채울 때는 `fillna(method='bfill')`

```
print(weather.loc['2012-02-11'])
print(weather.loc['2012-02-12'])
weather.fillna(method='ffill', inplace = True)
print(weather.loc['2012-02-10'])
print(weather.loc['2012-02-11'])
print(weather.loc['2012-02-12'])
print(weather[weather.isnull().any(axis=1)])
weather.describe()
```

```
평균기온    -0.7
최대풍속     NaN
평균풍속     NaN
Name: 2012-02-11, dtype: float64
평균기온     0.4
최대풍속     NaN
평균풍속     NaN
Name: 2012-02-12, dtype: float64
평균기온    -0.8
최대풍속     5.3
평균풍속     0.7
Name: 2012-02-10, dtype: float64
평균기온    -0.7
최대풍속     5.3
평균풍속     0.7
Name: 2012-02-11, dtype: float64
평균기온     0.4
최대풍속     5.3
평균풍속     0.7
Name: 2012-02-12, dtype: float64
```

시계열 자료 분석을 위한 DatetimeIndex

- **시계열** time series 데이터란 시간의 흐름에 따라 순차적으로 관측한 값들의 집합으로, weather 데이터프레임이 바로 시계열 데이터의 예이다.
- 판다스의 DatetimeIndex는 특정한 순간에 기록된 타임스탬프 형식의 **시계열** 자료를 다루기 위한 용도로 사용된다.
- DatetimeIndex의 사용법을 다음과 같은 예제 코드로 살펴보자. 아래와 같이 다양한 형식으로 표기된 연, 월, 일, 시간 데이터에 대하여 year, month, day, hour, minute 등과 같은 속성을 효과적으로 파싱하는 것을 볼 수 있을 것이다.

시계열 자료 분석을 위한 DatetimeIndex

▶ # 다양한 형식의 연, 월, 일 표시 데이터
d_list = ["01/03/2018", "01-03-2018", "2018-01-05", "2018/01/06"]
pd.DatetimeIndex(d_list).year # 년도 값을 출력함

Int64Index([2018, 2018, 2018, 2018], dtype='int64')

▶ pd.DatetimeIndex(d_list).month # 월 값을 출력함

Int64Index([1, 1, 1, 1], dtype='int64')

▶ pd.DatetimeIndex(d_list).day # 일 값을 출력함

Int64Index([3, 3, 5, 6], dtype='int64')

▶ dt_list = ["01,03,2018 11:12:13", "01-03-2018 11:22:13"]
pd.DatetimeIndex(dt_list).hour # 시 값을 출력함

Int64Index([11, 11], dtype='int64')

▶ pd.DatetimeIndex(dt_list).minute # 분 값을 출력함

Int64Index([12, 22], dtype='int64')

시계열 자료 분석을 위한 DatetimeIndex

- 우선 이 데이터에서 '일시'라는 이름의 열을 사용하여 연도와 달, 날을 얻기 위해서는 다음과 같은 방법을 사용하면 된다.



```
weather = pd.read_csv(weather_file, encoding='CP949')
weather['일시'] = pd.DatetimeIndex(weather['일시']).year
weather
```

	일시	평균기온	최대풍속	평균풍속
0	2010	28.7	8.3	3.4
1	2010	25.2	8.7	3.8
2	2010	22.1	6.3	2.9

<이하 생략>

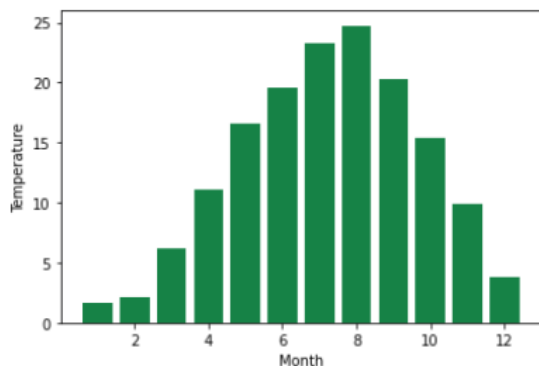
시계열 자료 분석을 위한 DatetimeIndex



```
... # 코드 생략
weather = pd.read_csv(weather_file, encoding='CP949')
weather['month'] = pd.DatetimeIndex(weather['일시']).month

monthly = [ None for x in range(12) ] # 달별로 구분된 12개의 None 값
monthly_wind = [ 0 for x in range(12) ] # 각 달의 평균 풍속을 담을 리스트
for i in range(12) :
    monthly[i] = weather[ weather['month'] == i + 1 ] # 달별로 분리
    monthly_wind[i] = monthly[i]['평균기온'].mean() # 개별 데이터 분석

months = np.arange(1, 13) # 1에서 12월의 연속된 수를 생성
plt.bar(months, monthly_wind, color='green')
plt.xlabel('Month')
plt.ylabel('Temperature')
```



	일시	평균기온	최대풍속	평균풍속	month
0	2010-08-01	28.7	8.3	3.4	8
1	2010-08-02	25.2	8.7	3.8	8
2	2010-08-03	22.1	6.3	2.9	8
3	2010-08-04	25.3	6.6	4.2	8
4	2010-08-05	27.2	9.1	5.6	8

특정한 값에 기반하여 데이터를 묶어보자: 그룹핑

- 데이터 분석을 할 때에는 특정한 값에 기반하여 데이터를 그룹으로 묶는 일이 많다. 이를 위해서 `groupby()`라는 메소드를 사용할 수 있다.
- 이제 'month' 라는 이름을 가지는 새로운 열을 만들어 보자.



새로운 'month' 열을 만든다

```
weather['month'] = pd.DatetimeIndex(weather['일시']).month  
print(weather)
```

	일시	평균기온	최대풍속	평균풍속	month
0	2010-08-01	28.7	8.3	3.4	8
1	2010-08-02	25.2	8.7	3.8	8
2	2010-08-03	22.1	6.3	2.9	8
3	2010-08-04	25.3	6.6	4.2	8

<이하 생략>

특정한 값에 기반하여 데이터를 묶어보자: 그룹핑

- 새로운 'month' 열에 대하여 다음과 같이 `groupby()`를 적용하여 보자. 이전 절과 달리 그룹으로 묶기 위해서 별도의 `for` 문을 사용할 필요가 없다. 그리고 여기에 `mean()`을 적용하면 해당 그룹의 데이터들이 가진 값의 평균을 구할 수 있다.
- 비수치형 자료형을 가진 일시 열을 계산에서 제외시키기 위해 `numeric_only = True`라는 인자를 준다.



```
# numeric_only = True라는 키워드 인자를 통해서 일시 열을 계산에서 제외  
# 'month'를 기준으로 묶고 평균을 구함  
monthly_means = weather.groupby('month').mean(numeric_only = True)  
print(monthly_means)
```

	평균기온	최대풍속	평균풍속
month			
1	1.598387	8.158065	3.757419
2	2.136396	8.225357	3.946786
3	6.250323	8.871935	4.390291
4	11.064667	9.305017	4.622483
5	16.564194	8.548710	4.219355

<이하 생략>

특정한 값에 기반하여 데이터를 묶어보자: 그룹핑

- 이전에 다룬 DatetimeIndex와 데이터프레임의 groupby() 기능을 이용하여 2010년도 8월부터 2020년도 7월까지 울릉도 지역의 평균기온, 최대풍속, 평균풍속의 평균값을 알아보자.
- 이를 위하여 다음과 같이 'year' 열을 만들어보자.



```
weather_file = './weather.csv'

weather = pd.read_csv(weather_file, encoding='CP949')

weather['month'] = pd.DatetimeIndex(weather['일시']).month
print(weather)
```

	일시	평균기온	최대풍속	평균풍속	year
0	2010-08-01	28.7	8.3	3.4	2010
1	2010-08-02	25.2	8.7	3.8	2010
2	2010-08-03	22.1	6.3	2.9	2010

<이하 생략>

특정한 값에 기반하여 데이터를 묶어보자: 그룹핑

- 평균값을 출력해 보자.
이제 월별 평균이 아닌 연도별 평균기온, 최대풍속, 평균풍속을 얻을 수 있다.
2010년도의 자료는 8월 이후의 날씨 자료만 있으며, 2020년도의 자료도 7월 31일까지만 있다는 점에 주의하자.



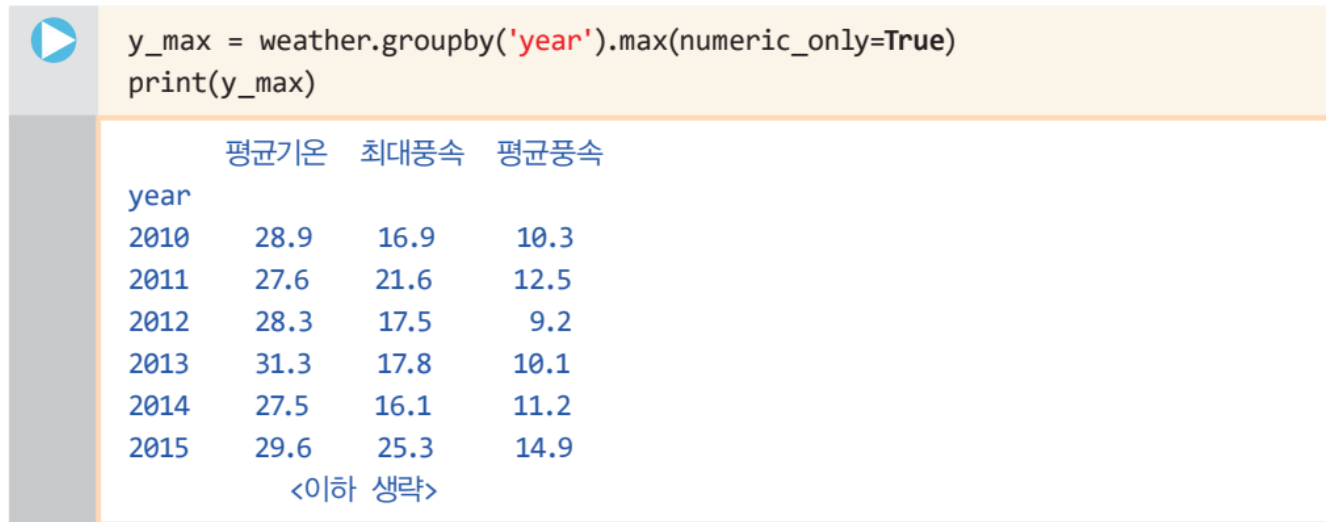
```
y_means = weather.groupby('year').mean(numeric_only=True)
print(y_means)
```

	평균기온	최대풍속	평균풍속
year			
2010	15.238562	8.205229	4.069281
2011	12.073425	8.355616	4.251233
2012	11.892896	7.794490	3.863912

<이하 생략>

특정한 값에 기반하여 데이터를 묶어보자: 그룹핑

- 울릉도 지역의 10년간 연간 최고 평균기온, 최대풍속, 평균풍속은 어떻게 알 수 있을까? 평균을 의미하는 `mean()` 대신 최고를 의미하는 `max()`를 사용해 보자.



The image shows a code execution environment. At the top, a code cell contains the following Python code: `y_max = weather.groupby('year').max(numeric_only=True)` and `print(y_max)`. Below the code, the output is displayed as a table with three columns: '평균기온' (Average Temperature), '최대풍속' (Maximum Wind Speed), and '평균풍속' (Average Wind Speed). The rows represent the years from 2010 to 2015. The values are: 2010 (28.9, 16.9, 10.3), 2011 (27.6, 21.6, 12.5), 2012 (28.3, 17.5, 9.2), 2013 (31.3, 17.8, 10.1), 2014 (27.5, 16.1, 11.2), and 2015 (29.6, 25.3, 14.9). At the bottom of the output, there is a link '<이하 생략>' (Omit the following).

	평균기온	최대풍속	평균풍속
year			
2010	28.9	16.9	10.3
2011	27.6	21.6	12.5
2012	28.3	17.5	9.2
2013	31.3	17.8	10.1
2014	27.5	16.1	11.2
2015	29.6	25.3	14.9

<이하 생략>

- 비록 2010년 7월 31일까지의 정보가 빠져있기는 하지만 2010년도에 가장 기온이 높은 날은 하루 평균 기온이 28.9도라는 것을 알 수 있으며, 2013년도의 가장 기온이 높은 날은 하루 평균 기온이 31.3도에 이르는 매우 무더운 날이 있었다는 것도 알 수 있다.

그룹핑과 필터링

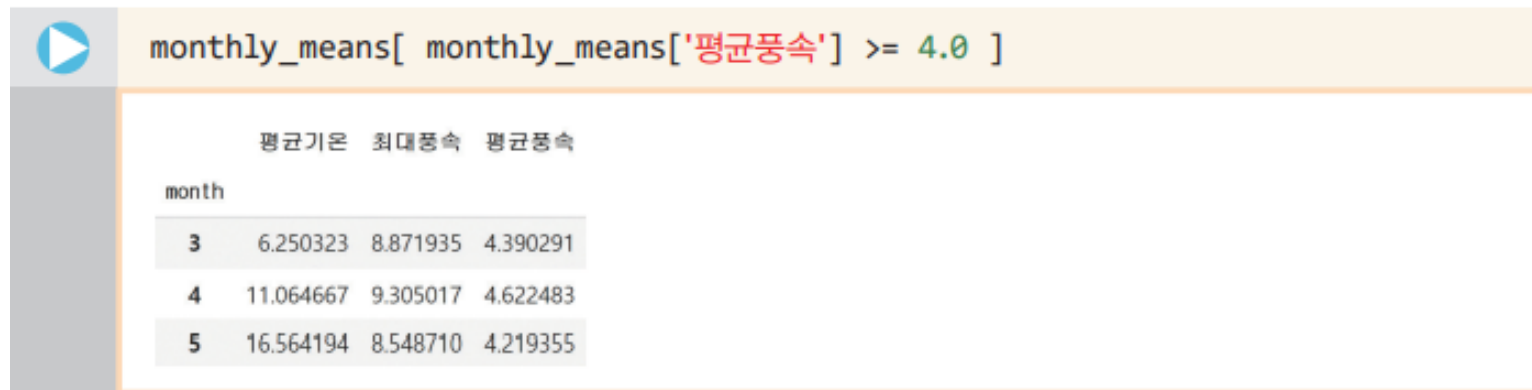
- 다음으로 필터링에 대하여 알아보자. 판다스는 특정한 조건을 주어서 데이터프레임의 값을 필터링할 수 있다.
- 예를 들어서 앞에서 만든 monthly_means 데이터프레임에서 평균 풍속이 4.0이 넘는지를 찾는 방법은 다음과 같다.

```
▶ monthly_means['평균풍속'] >= 4.0
```

month	
1	False
2	False
3	True
4	True
5	True
6	False
7	False
....<중간 생략>	

그룹핑과 필터링

- 이 코드를 보니 3, 4, 5월의 평균풍속만 4m/s 이상이며, 나머지는 그 미만임을 알 수 있다.
- 특정 조건을 만족하는 데이터만을 추출하는 방식은 이 조건을 활용하여 논리 인덱싱을 다음과 같이 수행한다.
- 울릉도에는 3, 4, 5월에 평균풍속이 4m/s 이상으로 바람이 많이 부는 것을 확인할 수 있다.



```
monthly_means[ monthly_means['평균풍속'] >= 4.0 ]
```

	평균기온	최대풍속	평균풍속
month			
3	6.250323	8.871935	4.390291
4	11.064667	9.305017	4.622483
5	16.564194	8.548710	4.219355

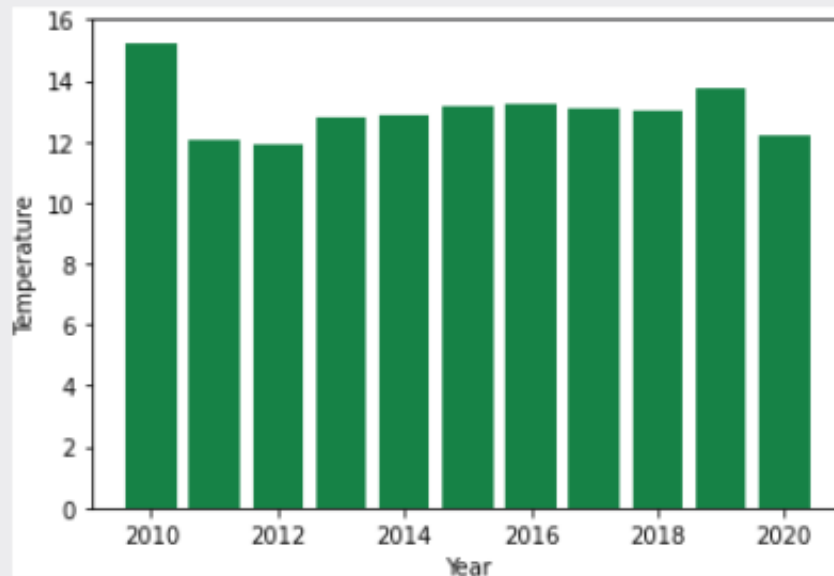
그룹핑과 필터링



히스토그램으로 연도별 평균기온을 표현하자

난이도 중

이 weather 데이터프레임을 이용하여 다음과 같이 연도별 평균기온을 히스토그램으로 출력하여라(힌트: `groupby()` 함수와 `DatetimeIndex().year`를 사용해보자. 히스토그램은 `plt.bar()` 또는 데이터프레임의 `plot(kind='bar')`로 그려보자).



데이터 구조를 변경하는 pivot()

- 판다스에서 이와 같이 테이블의 구조를 변경하는 대표적인 메소드가 바로 pivot()이다.
- 이 df 데이터프레임을 상품의 재질을 기준으로 다시 인덱싱하고자 한다.



```
df = pd.DataFrame({'상품' : ['시계', '반지', '반지', '목걸이', '팔찌'],  
                  '재질' : ['금', '은', '백금', '금', '은'],  
                  '가격' : [500000, 20000, 350000, 300000, 60000]})
```

df

	상품	재질	가격
0	시계	금	500000
1	반지	은	20000
2	반지	백금	350000
3	목걸이	금	300000
4	팔찌	은	60000

데이터 구조를 변경하는 pivot()

- 이때 사용된 pivot() 메소드에서 index='상품'을 사용하여 상품을 인덱스로 지정하도록 하자.
- 또한, columns='재질' 인자를 사용하여 재질을 열로하는 새로운 데이터프레임을 만들 수 있다.
- 이 데이터프레임은 가격을 값으로 가지는데 이 값들 중에서 존재하지 않는 항목은 fillna() 메소드를 사용해서 0으로 채워주었다.

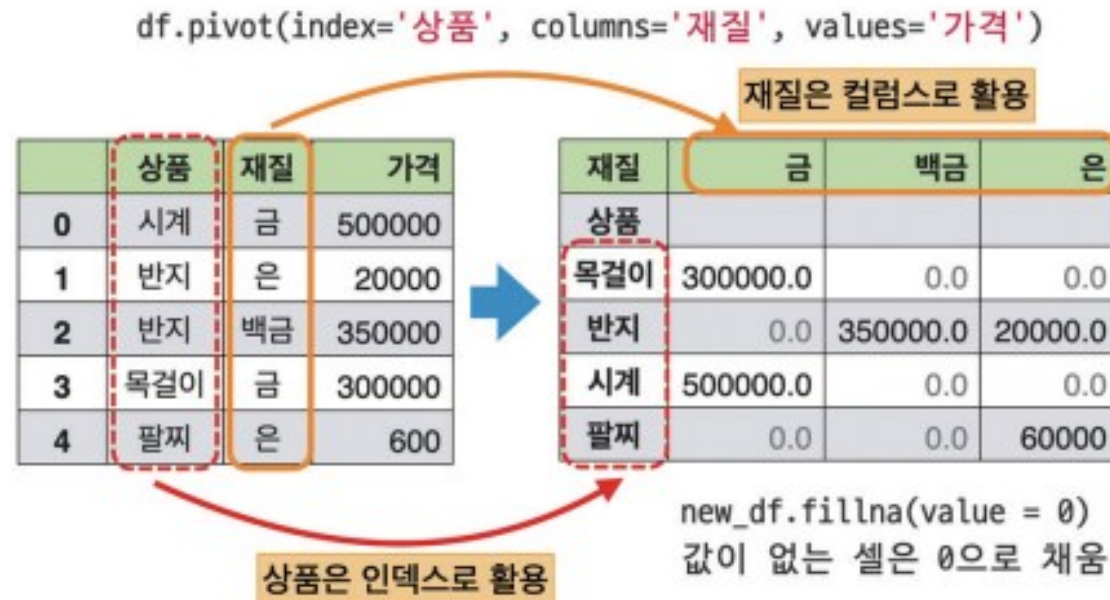


```
new_df = df.pivot(index='상품', columns='재질', values='가격')
new_df.fillna(value=0)
```

	금	백금	은
상품			
목걸이	300000.0	0.0	0.0
반지	0.0	350000.0	20000.0
시계	500000.0	0.0	0.0
팔찌	0.0	0.0	60000.0

데이터 구조를 변경하는 pivot()

- 이와 같이 구조를 변경하게 되면, 다음과 같이 재질 컬럼스의 값인 '금', '백금', '은' 각각이 새로운 컬럼 값이 되며 내부는 가격이라는 열의 값으로 채워지게 된다.



데이터 구조를 변경하는 pivot()

- 위의 pivot() 메소드와 유사한 pivot_table() 이라는 메소드를 통해서 데이터를 재구조화하는 것도 가능하다.
- 하지만 인덱스가 2개 이상인 경우 pivot() 메소드로는 재구조화가 안되며 pivot_table()을 사용해야만 한다.
- 마찬가지로 column이 2개 이상인 경우 pivot() 메소드로는 재구조화가 안되며 pivot_table()을 사용해야만 한다.

데이터 구조를 변경하는 방법으로 pivot()과 pivot_table()이 있습니다. pivot_table() 메소드를 사용하여 좀 더 고급스러운 재구조화를 할 수 있지요.



데이터 구조를 변경하는 pivot()



데이터프레임의 구조를 고쳐보자

난이도 상

다음 그림의 왼쪽과 같은 데이터프레임을 만들어라. 이 데이터프레임은 자동차의 이름, 브랜드, 종류, 가격이라는 열을 가지고 있다. 이 데이터프레임을 `pivot()` 메소드를 사용하여 오른쪽과 같은 데이터프레임으로 변환하여라.

	자동차명	브랜드	종류	가격
0	GV80	제네시스	내연기관	8000
1	IONIQ 5	현대자동차	전기차	5500
2	Model Y	테슬라	전기차	5600
3	IX3	BMW	전기차	8300
4	S Class	Mercedes Benz	내연기관	14000
5	GV60	제네시스	전기차	7000



자동차명	GV60	GV80	IONIQ 5	IX3	Model Y	S Class
브랜드						
BMW	0.0	0.0	0.0	8300.0	0.0	0.0
Mercedes Benz	0.0	0.0	0.0	0.0	0.0	14000.0
제네시스	7000.0	8000.0	0.0	0.0	0.0	0.0
테슬라	0.0	0.0	0.0	0.0	5600.0	0.0
현대자동차	0.0	0.0	5500.0	0.0	0.0	0.0

두 개의 데이터프레임을 하나로 합치는 concat()

- 일반적으로 데이터들은 작은 테이블로 나누어져 있는 경우가 많다.
이러한 데이터를 하나로 합치는 방법 가운데 하나인 concat() 함수를 살펴보자.
- 이 두 개의 데이터프레임을 결합하여 하나의 데이터프레임으로 만드는 방법에 대해 알아볼 것이다.

```
▶ df_1 = pd.DataFrame( {'A' : ['a10', 'a11', 'a12'],  
                        'B' : ['b10', 'b11', 'b12'],  
                        'C' : ['c10', 'c11', 'c12']} ,  
                        index = ['가', '나', '다'] )  
  
df_2 = pd.DataFrame( {'B' : ['b23', 'b24', 'b25'],  
                        'C' : ['c23', 'c24', 'c25'],  
                        'D' : ['d23', 'd24', 'd25']} ,  
                        index = ['다', '라', '마'] )
```

두 개의 데이터프레임을 하나로 합치는 concat()

- 이 두 데이터프레임의 결합을 위해 concat() 함수를 사용하면 된다.
- 이 함수의 첫 인자는 데이터프레임의 리스트이다.

	df_1		
	A	B	C
가	a10	b10	c10
나	a11	b11	c11
다	a12	b12	c12

인덱스

	df_2		
	B	C	D
다	b23	c23	d23
라	b24	c24	d24
마	b25	c25	d25

인덱스

- 그리고 두 개의 중요한 키워드 매개변수 axis와 join의 의미가 그림에 설명되어 있다. 레이블(열 이름)의 합집합, 교집합으로 합칠 수 있다.



두 개의 데이터프레임을 하나로 합치는 concat()

- df_1과 df_2의 결합을 위하여 concat() 함수의 인자로 합칠 데이터프레임 [df_1, df_2]를 인자로 주었다.



```
df_3 = pd.concat( [df_1, df_2])  
print(df_3)
```

	A	B	C	D
가	a10	b10	c10	NaN
나	a11	b11	c11	NaN
다	a12	b12	c12	NaN
다	NaN	b23	c23	d23
라	NaN	b24	c24	d24
마	NaN	b25	c25	d25

두 개의 데이터프레임을 하나로 합치는 concat()

- concat() 함수에서 별다른 인자를 주지 않을 경우 디폴트 axis=0을 사용하여 행을 늘려서 테이블을 붙여간다.
- 따라서 다음과 같은 특징을 가지는 새로운 테이블이 만들어졌다.

	A	B	C	D
가	a10	b10	c10	NaN
나	a11	b11	c11	NaN
다	a12	b12	c12	NaN
다	NaN	b23	c23	d23
라	NaN	b24	c24	d24
마	NaN	b25	c25	d25

인덱스 중복

- 두 테이블이 각각 3개의 행을 가지므로, 모두 6개의 행을 가지는 테이블이 생성.
- 인덱스 '다'는 두 데이터프레임에 모두 존재하므로 별도의 행으로 중복됨.
- 'outer'를 디폴트 결합 방식으로 사용하여, A, B, C와 B, C, D의 합집합인 A, B, C, D가 최종적인 열이 됨.

두 개의 데이터프레임을 하나로 합치는 concat()

- 만일 다음과 같이 `join='inner'`를 인자로 줄 경우 두 테이블의 교집합인 B, C 열이 최종 열이 되는 것도 확인할 수 있다.



```
df_4 = pd.concat( [df_1, df_2], join='inner')  
print(df_4)
```

	B	C
가	b10	c10
나	b11	c11
다	b12	c12
다	b23	c23
라	b24	c24
마	b25	c25

테이블 데이터의 결합: concat()과 merge()

- concat()는 판다스의 함수로 결합을 위하여 하나 이상의 데이터프레임을 인자로 받는다.
- 반면 merge()는 데이터프레임의 메소드이며, 두 데이터프레임을 각 데이터에 존재하는 고유값(key)을 기준으로 병합할때 사용한다.

`DataFrame.merge(right, how='inner', on=None)`

`right` : 현재의 데이터프레임과 결합할 데이터프레임

`how` : 결합의 방식 'left', 'right', 'inner', 'outer' 가능

`on` : 조인 연산을 수행하기 위해 사용할 레이블(두 데이터프레임 모두에 존재해야 함)

테이블 데이터의 결합: concat()과 merge()

- on='B'를 이용하여 'B' 레이블을 가진 열의 데이터를 키로 조인 연산을 하였다.
- how 매개변수에는 아래와 같이 네 종류의 인자를 넘길 수 있다.
- on='B'를 유지한 상태로 how를 변경하여 다양한 결과를 확인해 보라.
- 'left'는 왼쪽 df 기준으로 모든 행유지 및 오른쪽에서 매칭되는것만 가져옴.



```
print('left outer \n' , df_1.merge(df_2, how='left', on='B' ) )
print('right outer \n' ,df_1.merge(df_2, how='right', on='B' ) )
print('full outer \n' ,df_1.merge(df_2, how='outer', on='B' ) )
print('inner \n' ,df_1.merge(df_2, how='inner', on='B' ) )
```

C_x : df1에서 온 C
C_y : df2에서 온 C

left outer

	A	B	C_x	C_y	D
0	a10	b10	c10	NaN	NaN
1	a11	b11	c11	NaN	NaN
2	a12	b12	c12	NaN	NaN

right outer

	A	B	C_x	C_y	D
0	NaN	b23	NaN	c23	d23
1	NaN	b24	NaN	c24	d24
2	NaN	b25	NaN	c25	d25

full outer

	A	B	C_x	C_y	D
0	a10	b10	c10	NaN	NaN
1	a11	b11	c11	NaN	NaN
2	a12	b12	c12	NaN	NaN
3	NaN	b23	NaN	c23	d23
4	NaN	b24	NaN	c24	d24
5	NaN	b25	NaN	c25	d25

inner

Empty DataFrame

테이블 데이터의 결합: concat()과 merge()

- 두 데이터프레임에서는 'B'뿐만이 아니라 'C' 레이블도 두 테이블에 동시에 나타난다.
- 이럴 경우 왼쪽 테이블에서 가져온 것은 'C_x', 오른쪽에서 가져온 것은 'C_y'로 이름을 새로 붙여 테이블을 만든다.



```
df_3 = df_1.merge(df_2, how='outer', on='B')  
print(df_3)
```

	A	B	C_x	C_y	D
0	a10	b10	c10	NaN	NaN
1	a11	b11	c11	NaN	NaN
2	a12	b12	c12	NaN	NaN
3	NaN	b23	NaN	c23	d23
4	NaN	b24	NaN	c24	d24
5	NaN	b25	NaN	c25	d25

테이블 데이터의 결합: concat()과 merge()

- merge() 함수의 동작 결과를 살펴보면, 원래 테이블에 있던 인덱스는 사라진 것을 확인할 수 있다.
- 많은 경우 인덱스가 키의 역할을 수행하는 경우도 있다. 이런 경우에 인덱스를 키로 사용하라고 할 수도 있다.

on = 'B'

	A	B	C_x	C_y	D
0	a10	b10	c10	NaN	NaN
1	a11	b11	c11	NaN	NaN
2	a12	b12	c12	NaN	NaN
3	NaN	b23	NaN	c23	d23
4	NaN	b24	NaN	c24	d24
5	NaN	b25	NaN	c25	d25

left

right

outer: b10, b11, b12, b23, b24, b25

inner: 교집합 없음
{b10, b11, b12}
{b23, b24, b25}

테이블 데이터의 결합: concat()과 merge()

- 양쪽 테이블의 인덱스를 모두 사용하여 병합의 키가 되도록 하는 방식은 아래와 같다. 양쪽 테이블 중 하나라도 인덱스가 있으면 결과에 포함시킨다.
- 'outer' 방식이므로 두 인덱스의 합집합이 새로운 인덱스가 된다.
- 'inner'라면 교집합으로 '다' 행만 남게 된다.



```
df_3 = df_1.merge(df_2, how = 'outer',  
                  left_index = True, right_index = True )  
print(df_3)
```

	A	B_x	C_x	B_y	C_y	D
가	a10	b10	c10	NaN	NaN	NaN
나	a11	b11	c11	NaN	NaN	NaN
다	a12	b12	c12	b23	c23	d23
라	NaN	NaN	NaN	b24	c24	d24
마	NaN	NaN	NaN	b25	c25	d25